

CS24: Binary Heap Lecture Notes

Motivation

Real-world problem → Algorithm → ADT → Data structure → Representation

Navigation	Dijkstra's	Priority Queue	Binary Heap	Vector
Data Compression	Huffman Coding	Priority Queue	Binary Heap	Vector

Last lecture we used `priority_queue` as a black box. Its run times:

Operation	Run time
<code>top()</code>	$O(1)$
<code>push(x)</code>	$O(\log n)$
<code>pop()</code>	$O(\log n)$ — ?

And unlike a BST (non-self balancing kind like the one you implemented) — where inserting sorted data degenerates the tree into a linked list and collapses search to $O(n)$ — a binary heap **always** guarantees these run times. No degenerate case.

Question: Why is `top()` $O(1)$ but `push()` and `pop()` are $O(\log n)$?

The diagram shows two binary trees. The left tree is a full binary tree with 7 nodes, where every level is completely filled. It is labeled "Complete ✓". The right tree is a binary tree with 7 nodes, but the root's right child is missing, making it not a complete binary tree. It is labeled "Not complete".

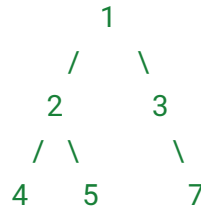
Complete ✓ Not complete

- (structure!)

```

graph TD
    1((1)) --> 2((2))
    1((1)) --> 3((3))
    2((2)) --> 4((4))
    2((2)) --> 5((5))
    3((3)) --> 6((6))
    3((3)) --> 7((7))
    4((4)) --> 20((20))
  
```

```
(missing left child,
  has right child)
```



(last level not
left-filled)



2. Binary Heap: Two Properties

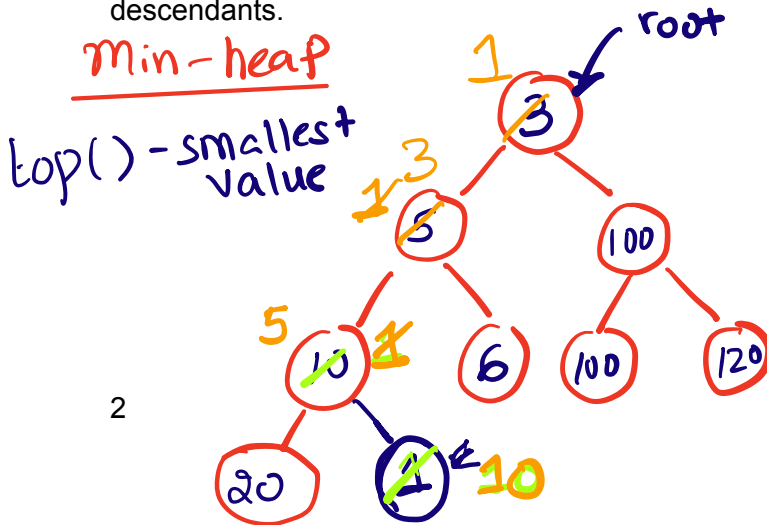
(1) Shape property: The tree is a complete binary tree.

Case (1)

- **Max-heap:** for every node x , $\text{key}(x) \geq \text{key}(\text{children of } x)$

This is why `top()` is $O(1)$ — the answer is always sitting right there.

Min-heap



push (1)

→ insert key(i) at the first open spot → $O(1)$

→ Bubble up

$1 < 10 \rightarrow \text{swap}$

$\frac{1}{2} < 5 \rightarrow \text{swap}$

$\frac{1}{2} < 3 \rightarrow \text{swap}$

3. Heap Operations

$\text{top}()$ — $O(1)$

None

```
procedure top():  
    return root.key
```

$\text{push}(x)$ — $O(\log n)$

None

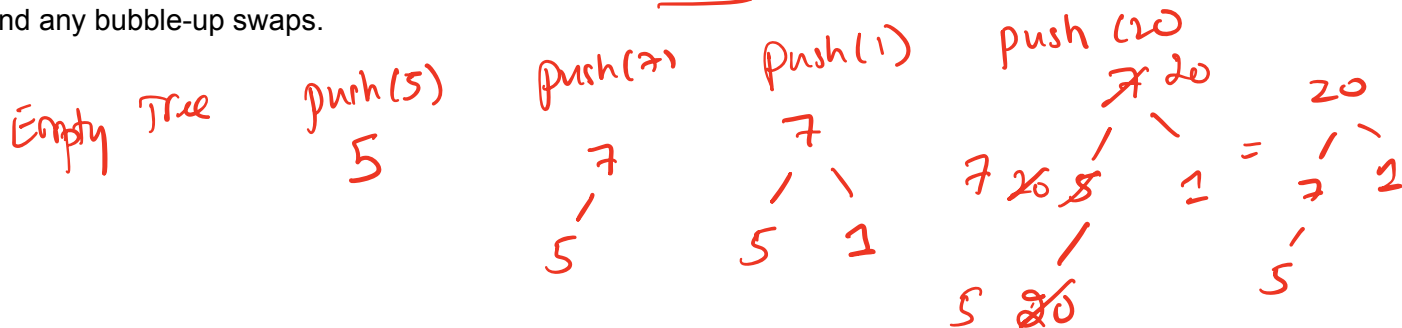
```
procedure push(x):  
    1. Insert x in the first open spot in the tree  
    2. while x has a parent AND parent(x) < x:    ← max-heap condition  
        swap(x, parent(x))                        ← bubble up
```

Why $O(\log n)$? Step 2 bubbles up at most one level per swap, and the tree height is $\lceil \log_2 n \rceil$. So at most $O(\log n)$ swaps.

⚠ **Open question — hold this:**

Step 1 says "insert in the first open spot in $O(1)$." On a tree with node pointers, finding that spot would require traversal. *How can this possibly be $O(1)$?*

Activity 1: Insert 5, 7, 1, 20 into an initially empty max-heap. After each insertion, show the tree and any bubble-up swaps.



`pop()` — $O(\log n)$

None

procedure `pop()`:

Step 1: Replace key of root with key of last node in the last level

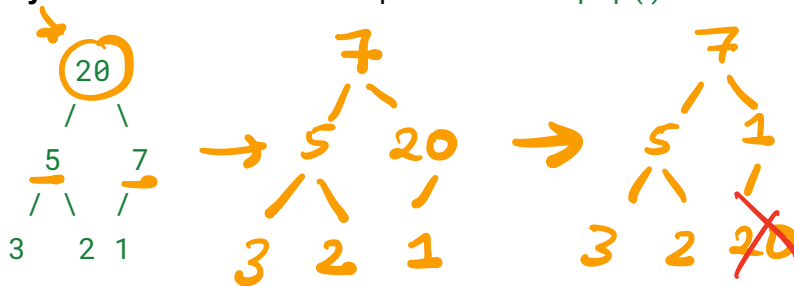
Step 2: Starting at root, "bubble down" until heap property is restored

Bubble down (max-heap): swap with the **larger** child until heap property holds. **Bubble down (min-heap):** swap with the **smaller** child until heap property holds.

Why Step 1? If we just deleted the root, the tree would no longer be complete. By replacing it with the last node, we preserve the shape property in $O(1)$ — then restore the heap property by bubbling down.

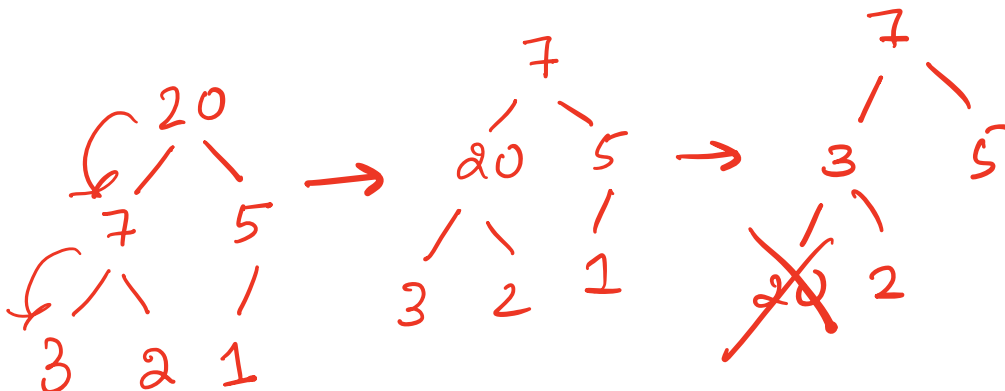
Why $O(\log n)$? Each bubble-down swap moves one level down. Height = $\lfloor \log_2 n \rfloor$, so at most $O(\log n)$ swaps.

Activity 2: Start with the max-heap below. Trace `pop()`.

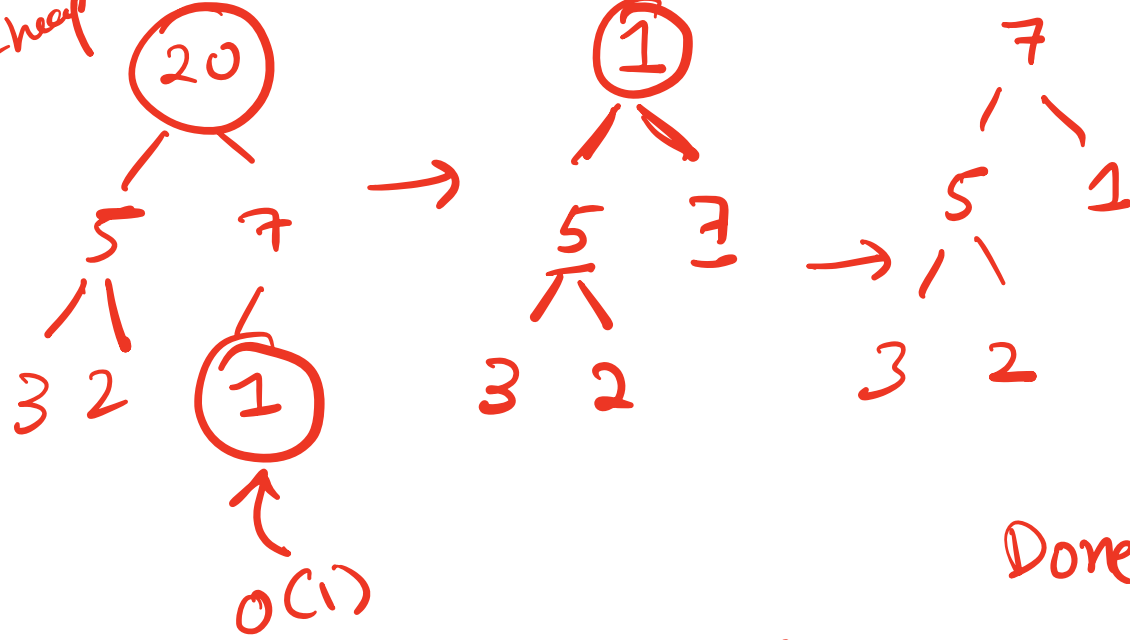


Bubble down, delete the node

Counter example → this approach has a flaw



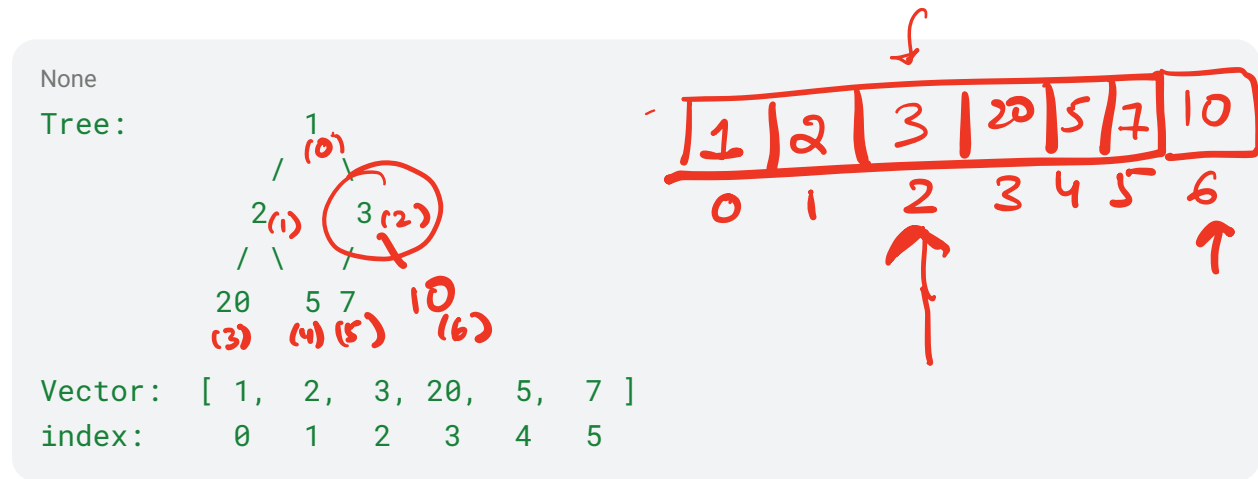
max-heap



Step 1. → Replace 20 by 1
Step 2. → Bubble down

4. The Reveal: Vector Representation

A complete binary tree can be stored as a **plain vector** using level-order (BFS) order. No node pointers. No tree struct. Just a vector.



Index formulas (0-indexed):

	Formula
Root	index 0
Parent of node at index i	$\lfloor (i - 1) / 2 \rfloor$ $i=6 \rightarrow \lfloor (6-1)/2 \rfloor = 2$
Left child of node at index i	$2i + 1$
Right child of node at index i	$2i + 2$

Quick checks — answer these on the vector above:

Question	Answer
What is the root?	key = <u>1</u> at index <u>0</u>
Where is the next open spot for a new insertion?	index <u>6</u>
Parent of node at index 4 (key 5)?	index <u>1</u> , key <u>2</u> $\text{parent}(4) = \lfloor \frac{4-1}{2} \rfloor = 1$
Children of node at index 1 (key 2)?	left: index <u>$2 \cdot 1 + 1 = 3$</u> right: index <u>$2 \cdot 1 + 2 = 4$</u>

Activity 3: Bubble-up in `push()` moves a newly inserted node toward the root — let's make sure you can trace that path on the vector.

Given `[1, 2, 3, 20, 5, 7]`, a new key was just inserted at index 5 (key 7). Apply the parent formula repeatedly to trace the path from index 5 to the root.

Sequence of indices: _____

None

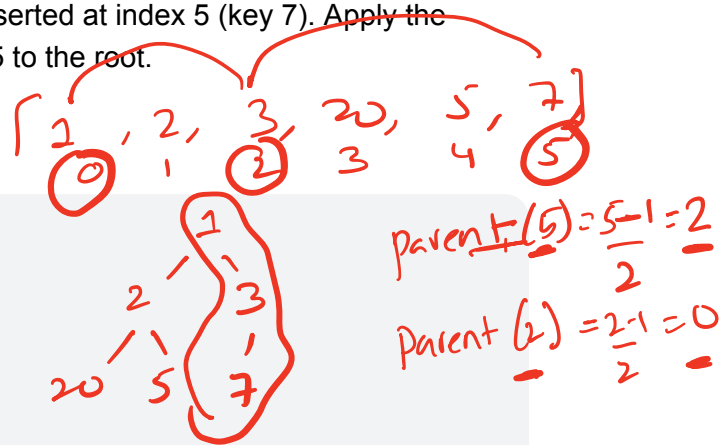
A. 5, 4, 3, 2, 1, 0

B. 5, 4, 2, 1, 0

C. 5, 3, 1

D. 5, 2, 0 (correct)

E. None of these



Answer the open question:

The next open spot is always the **next index in the vector** — just `v.push_back(x)`.

Traversing up/down is just arithmetic — no pointer chasing:

None

Bubble up: $i \rightarrow \lfloor (i-1)/2 \rfloor$

repeat until heap property holds

Bubble down: $i \rightarrow 2i+1$ or $2i+2$

pick the better child, repeat

Each step is $O(1)$. At most $\lfloor \log_2 n \rfloor$ steps. **That's where $O(\log n)$ comes from.**

The structure IS what makes the algorithm fast.

How many leaf nodes in a complete binary tree with n nodes?
 Approx. $\frac{n}{2}$ because index of last leaf node is $n-1$. Index of its parent is $\frac{(n-1)-1}{2} = \frac{n}{2} - 1$. Total leaf = $n - (\frac{n}{2} - 1) = \frac{n}{2} + 1$.

Why start at $n/2 - 1$? Leaves are at indices $n/2$ through $n-1$. They trivially satisfy the heap property — skip them. Start at the last non-leaf and work up to the root.

Why is this $O(n)$ and not $O(n \log n)$? Based on analysis on previous page. The buildHeap also fixes the heap property of nodes at the lower levels of the tree, working upwards. Leaf nodes require no work. Nodes closer to the leaf nodes require fewer swaps & there are many more nodes at the lower levels. A more careful analysis gives $O(n)$.

The key insight: most nodes are near the bottom and bubble down very little.

Batch 0 (leaves, indices $n/2$ to $n-1$): $n/2$ nodes, 0 swaps each — not in the loop.

Batch ⚡	loop variable i	# iterations = # nodes on which bubbleDown() is called	max swaps to bubble down
1	$n/4$ to $n/2-1$	$n/4$	1
2	$n/8$ to $n/4-1$	$n/8$	2
b	$n/2^{b+1}$ to $n/2^b - 1$	$n/2^{b+1}$	b ←
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\lfloor \log n \rfloor$	$i = 0$ (root)	1	$\lfloor \log n \rfloor$

Total swaps across the entire buildHeap:

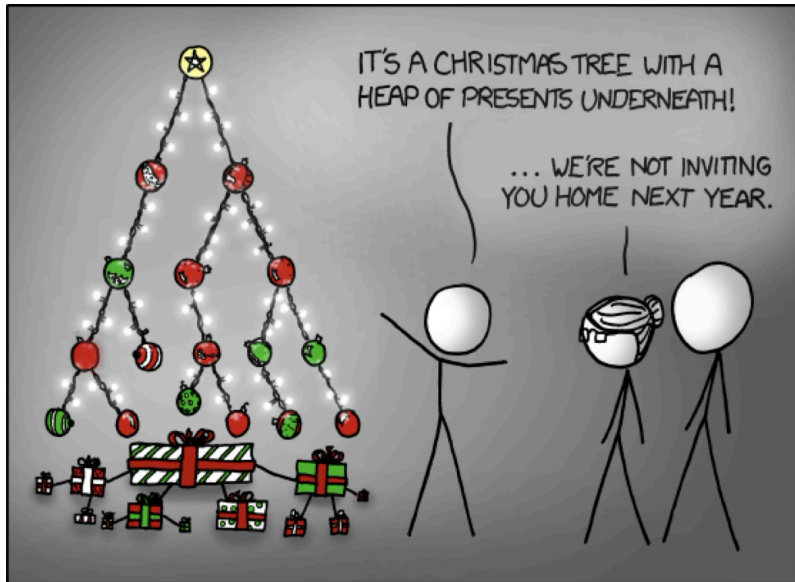
None

$$T(n) = c \cdot \sum_{b=0}^{\log n} [n/2^{b+1} \cdot b]$$

$$= c \cdot n \cdot \sum_{b=0}^{\log n} [b / 2^{b+1}]$$

$$= c \cdot n \cdot (\text{constant}) \quad \leftarrow \text{this sum converges to } \leq 2$$

$$= O(n)$$



Summary

ADT	Priority Queue
Data structure	Binary Heap
Underlying representation	Complete Binary Tree stored as a Vector
<code>top()</code>	$O(1)$ — root is always the best element
<code>push(x)</code>	$O(\log n)$ — append to vector + bubble up
<code>pop()</code>	$O(\log n)$ — swap with last + remove + bubble down
<code>buildHeap</code> (heapify)	$O(n)$ — most nodes are leaves and do zero work

- The **heap property** is what makes `top()` $O(1)$.
- The **complete binary tree shape** (stored as a vector) is what makes `push()` and `pop()` $O(\log n)$ — guaranteed, no degenerate case.

Next: Lab 04 — implement your own priority queue as a binary heap.